

1. Version 1

Only changed the print statement to show Store

```
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365/Version 1 - Basic Implementation$ gcc -o stack-v1 app.c
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365/Version 1 - Basic Implementation$ ./stack-v1
Popped : xyz
```

- Even though there was a pop call on the main method, the output of the stack is still complete with its pushes. This is due to the pop() function only decrementing the top index, but does not actually remove the elements in memory on the array. C behaves in a sense that when it parses an array to gather its elements, it iterates through its elements in memory until it reaches its null terminator (\0) to signify that the rest are empty slots and it ends here. Now, when the print function uses the %s format specifier, it tells the variable mentioned to gather and iterate its elements written in memory until it reaches the null terminator; hence, it still shows the complete xyz.

Followed the 5 chars push, 4 pops, 2 chars push, and print Store

```
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365$ cd "Version 1 - Basic Implementation"
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365/Version 1 - Basic Implementation$ gcc -o stack-v1 app.c
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365/Version 1 - Basic Implementation$ ./stack-v1
Popped : afgde
```

- Recalling the logic from above, the elements are still written in memory that somewhat overrides the top integer. So, it goes through the 5 pushes as usual; then, the top int index gets decremented to 1 with the pop calls. However, once it calls the last 2 pushes, it simply replaces the 2nd and 3rd element on the stack. The other elements won't be lost in the first place since the pop logic only manipulates the top integer but doesn't remove any element in memory.

2. Version 2

```
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365/Version 2 - Encapsulation and Modular Programming$ gcc -o stack-v2 app.c stack.c
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/Hw1_CSCI70_Castillo_231365/Version 2 - Encapsulation and Modular Programming$ ./stack-v2
Popped: y
```

- The three pushes perform as usual that leads the top int be equal to 3, with x, y, z stored at index 0, 1, and 2 respectively. Now with the manual manipulation of the top to 2, the tracker for getting the element gets overridden so that when pop() is now called, it will first decrement the top to 1 due to --top and return the element at that index, which is y.

3. Version 3

```
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/HM1_CSCI70_Castillo_231365/Version 3 - Structs with Pointers$ gcc -o stack-v3 app.c stack.c
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/HM1_CSCI70_Castillo_231365/Version 3 - Structs with Pointers$ ./stack-v3
Popped from s1: G
```

- With the problem from the slideset that the application being able to directly modify the stack, let us say that stack s1 had pushes on y and z. Since pointers are direct access to the memory block of their specific data, it allows the data to be manually overwritten so that it resets the top integer back to 0 and push a specific element like G in this case. This, in turn, give no security and encapsulation protection on the prior elements before as G was the sole output from the stack instead, overriding the past elements.

4. Version 4

```
compilation terminated.
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/HM1_CSCI70_Castillo_231365/Version 4 - Structs with Handlers$ gcc -o stack-v4 app.c stack.c
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/HM1_CSCI70_Castillo_231365/Version 4 - Structs with Handlers$ ./stack-v4
Pushing to a out-of-bound arbitrary stack.Push B to stack number 20...Invalid stack number 20
Pop B to that stack number 20 assuming it successfully pushed...Invalid stack number 20

Popped from stack 0: z
pc@DESKTOP-CSALOOM:/mnt/d/HanzoPC/Documents/HanzYr4/csci_70/HM1_CSCI70_Castillo_231365/Version 4 - Structs with Handlers$
```

- Following the problems from the slideset, the handle is artificial in a sense that it still compiles arbitrary and insensible values that are out of scope to the implementation. Hence, calling stack 20 still goes through and can be redundant in certain use cases given that its only an integer guard. Another would be the more complex implementations on the stack module, just for the sake of addressing these out-of-bound nonsensical inputs, a lot of boilerplate guards like the:

```
C/C++
if(stacknum >= 0 && stacknum < MAX_STACKS)
```

Moreover, the hardcode on the max stack is an issue since the cases where the application needs more than it will just throw logical errors and won't accommodate to it as there should be external manipulations to manually change it and this could be troublesome as time goes.

5. Version 5

- for encapsulation, if a programmer wants to write the stack in such sense that the main method for the user to interact with it is simply the push and pop functions. This can be their chosen way to go as this hides the implementation and backend works of the stack functions.

- if they want to enforce strict limits like there should only be 10 stacks, they might opt for the handlers approach so that to goes through guards before they compile and perform their actions in accordance to the code's goal and requirements

6. Version 6
 - Nothing to do here, just for the sake of showing in standard c++